Generating Test Cases and Procedures from Use Cases in Dynamic Software Product Lines

Italo L. Araújo [*]
Federal University of Ceará
Fortaleza, Brazil
italoaraujo@great.ufc.br

Ismayle S. Santos [*]
Federal University of Ceará
Fortaleza, Brazil
ismaylesantos@great.ufc.br

João B. Ferreira Filho
Federal University of Ceará
Fortaleza, Brazil
bosco@great.ufc.br

Rossana M. C. Andrade [†]
Federal University of Ceará
Fortaleza, Brazil
rossana@great.ufc.br

Pedro Santos Neto [†]
Federal University of Piauí
Teresina, Brazil
pasn@ufpi.edu.br

ABSTRACT

Software engineering is systematically evolving to address the production of families of systems instead of single products. This evolution comes at a price, it is now essential to deal with variability at both design and execution time. Developing test cases and procedures for a whole family of systems considering this dynamicity (variability at runtime) can be challenging. We propose a method to generate tests from use case specifications expressed in a controlled natural language, yet considering the variability and dynamicity in those specifications. We evaluate our method against use case specifications of a family of mobile and context-aware systems. Experimenting our method, we could measure that developing test cases and procedures becomes around 40% faster when using our approach, opposed to manual development of tests under the same conditions.

CCS Concepts

•**Software and its engineering** → **Software testing and debugging;** *Command and control languages;*

Keywords

Test generation; dynamic software product lines; context-aware

1. INTRODUCTION

Automation in requirements engineering has been studied increasingly in the past decades [13, 21]. Studies seek to provide methods and tools to facilitate the generation of software artifacts based on user's or domain expert's knowledge — often described in *use cases*. As use cases contain information about the behavior of part of a system, it can be used to generate tests. This generation is already challenging for one single system; it is a very complex task to generate it for a product line [6, 18]. In the case of a product line, the automatic generation of tests should take into account the variability information to generate a comprehensive set of tests (in this sense, many approaches have worked on strategies to enhance test coverage, like T-wise approaches [16, 11]). Also, the execution of a given test may be conditioned to the presence or activation of a set of features of the product line.

In some types of systems, it is possible to express their execution scenarios based on context information beforehand. This is particularly true in mobile, context-aware and dynamic systems [4, 5]; these systems can turn on/off parts (features) of itself to cope with a determined situation, e.g., Downloading a video may not be supported in case your mobile phone is on 3G. Therefore, in a product line of such systems (i.e., Dynamic Software Product Line [10]), in the requirements engineering phase, engineers can express the desired functionalities of the family of systems according to a given context. They do so by: creating informal descriptions in natural language or semi-structured spreadsheets. This informality challenges the automatic creation of tests or any executable artifacts.

- From use case descriptions, can we generate concrete test procedures and test cases?
- Can we also take into account the variability and dynamicity/context changes of the DSPL in this generation?

This paper seeks to answer these questions by proposing an approach to automatically generate test procedures and cases (i.e., input and output) for family of systems with dynamicity according to context information. We introduce a

[*]CAPES Scholarship
[†]CNPq Productivity Scholarship

SAC 2017,April 03-07, 2017, Marrakech, Morocco

http://dx.doi.org/10.1145/3019612.3019790

controlled natural language (CNL) to express textual information of use cases of DSPLs. Then, we take these descriptions as input to our test generation approach.

We evaluate the applicability of our approach by applying it in a family of mobile and context-aware systems. We translate the use cases of this family into descriptions in our CNL, which are then used as input to evaluate the efficacy and efficiency of the test generation in a controlled experiment with users of the test generation approach. We prove the applicability of the approach and verify that the test generation process becomes around 40% faster when using our method.

The remainder of this paper is divided as follows. Section 2 presents some preliminary information and motivation. Section 3 describes our two-fold approach: the CNL and the test generation method. Section 3, while Section 4 evaluates it with an experiment. Finally, Section 5 shows the related work and Section 6 concludes and gives perspectives about future work.

2. PRELIMINARIES AND MOTIVATION

The main goal of this work is to support the generation of test procedures and cases from use case descriptions. As depicted in Figure 1(a), our approach will use information from use case templates as input to generate both test cases (see Figure 1(b)) and test procedures (see Figure 1(c)). Test cases are the specification of the tests' input and output and the test procedures are the steps, coded in Java, performed during the test.

Part of the information of a use case template can be easily translated into elements in the test artifacts, like their own name, which are then used to also identify the tests. However, some data needs complex treatment in order to be useful during the test generation process. This is the case of natural language descriptions of execution steps. Figure 1(a) shows that part of the use case template contains these steps, e.g., "The System shows the plain text selected by the Visitor", which will trigger the generation of a test that will automatically click in a given text selection.

We follow efforts in the literature that try to define common artifacts for Software Product Line Engineering, specially the ones focused on dynamic and context-aware applications [7, 12, 17]. Therefore, the use case template used in this work [17] also contains the information about the variability and dynamicity of the use case. The use case is linked to a feature of the product line and also contains the optionality information (e.g., Mandatory in Figure 1(a)). As for the dynamic information, we have a field to describe the contextual rules that will trigger a dynamic reconfiguration, e.g., *LOW_BATTERY*.

Given the above scenario, our approach is two-fold: we first define a controlled natural language (CNL), called CARNAUbA (Controlled nAtuRal laNguAge for context-aware software product lines Use cAses), to express the use cases, and then we use these descriptions as input to a test generation method. The two parts of our approach are explained in the next section.

3. APPROACH

3.1 CNL

Creating a Controlled Natural Language to describe use cases in DSPLs allows us to place our work in the middle ground between Natural Language Processing [9] and Model to Text transformations [3]. We constrain the informal natural language so the machine can better (with less ambiguity) interpret it but still not going as far as if using a well-structured model, balancing the trade-off between expressiveness and machine-readability.We following show the CNL's (CARNAUbA) syntax and after, the semantics is described by means of the test generation method.

Syntax

According to [20], a CNL should be easy to learn, teach and understand, with minimum number of syntactical constructs. Following this tenet, CARNAUbA has 13 syntax elements; for the sake of readability, we show the 7 main syntactic constructs of the language in Table 1. For each syntax element in the table, we show the BNF-like definition and examples of valid instances in the language.

The element "Reuse Category" can range from "mandatory" if the feature must be in all products of the line, "alternative" if it may be chosen among other alternatives, to "optional". "Necessary Context", allows for expressing first order logic expressions like comparisons, conjunctions, etc. For example, we can write "temperature $>$ 50" or "HIGH_BATTERY" to indicate the contextual situation.

To describe the variation point and the variant (i.e., features), the user of the language needs to add initially the "[" symbol followed by the letters. After this, the variation point is finalized by "]".

An execution step in a Use Case expressed with CARNAUbA can be formed by three different elements: *Action*, *Conditional* and *Loop*. The *action* expresses the direct interaction of an actor with the system, or the systems' response to an actor. For example, "The User selects the option of the connexion context" and "The System displays the current visitor's indoor position". These steps are composed by a subject and a predicate; the subject must have an article and a nominal head (e.g.,"The User" and "The System"). It is also possible to set a post-nominal modifier to further specify the actor (e.g., The System Admin).

The predicate can have the word "no" to indicate negation of an action. In this kind of step, it is mandatory to have a verb and, at least, one complement. The verb must be in our list of supported verbs (e.g., show, choose, add, delete, etc). Each verb has only one situation to be used in order to avoid ambiguity. The full list of supported verbs can be found in the paper's web page [1]; we explain some of them and their semantics in the next subsection in Table 2.

The *conditional* allows the description of an IF action through an event. For example, "If the data authentication is valid, the system displays the message 'Welcome to this lab"'. The action to take is to display a welcome message to

[1]https://sites.google.com/site/use2testswithcnl/

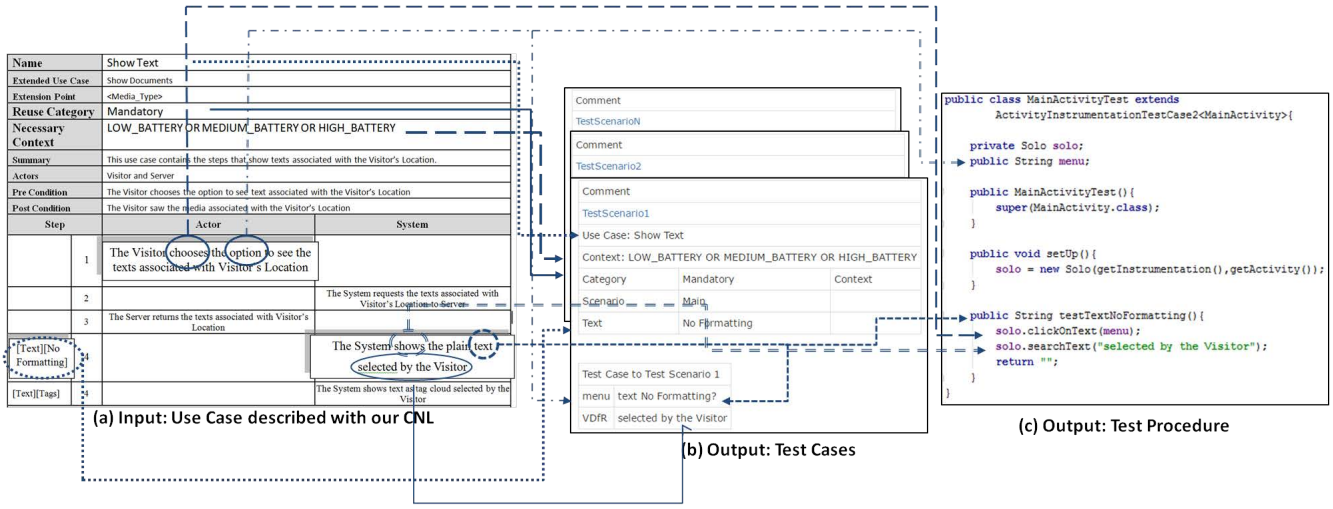

Figure 1: Input and output of our approach.

Table 1: Main syntax elements of our CNL for defining use cases of DSPLs.

	Use Case Name	Reuse Category	Necessary Context	Variation Point and Variant	Action	Conditional	Loop
BNF Description	Use Case: Name (ID | INT) * Name: ('A'..'Z') ('a'..'z'|'A'..'Z'|'0'..'9' |'_'|'-') *	Category: "Mandatory" | "Alternative" | "Optional"	Necessary Context: '(' variable (signal val)? exp? ')' (exp)? | variable (signal val)? (exp)? exp: (unary|binary) Context Constraint unary: 'and' | 'or' binary: '!' variable: ID ID* signal: '<' | '>' | '<=' | '>=' | '=' val: ID | INT	Variation Point: "[" ID ID* "]" Variant Variant: "["ID ID* "]"	Sentence: Subject Predicate "." Subject: Article Core_Subject (Complement)? Article: "The"| Indefinite_Article Indefinite_Article: "A" |" An" Core_Subject: ID_SUBJECT* Complement: ID_COMPLEMENT * Predicate: Verb Complement_Predicate Verb: ID_VERB* Complement_Predicate: ID_COMPLEMENT *	Conditional: "If" Condition"," "then" Action Condition: ID ID*	Loop: "Repeat" ("the" "step" INT | "the" "steps" INT to INT) (INT "times" | "until" condition)
Example	Authentication Access to the Context Show Documents Show_Texts	Mandatory Alternative Optional	HIGH_BATTERY temperature >50 (temperature >50) and (humidity >30)	[Show Documents][Show Texts]	The User types the login. The System shows a success message. The User types the login and the password.	If temperature >50, then the System shows a alert message.	Repeat the step 3 10 times. Repeat the steps 3 to 5 until the System identifies the user's location.

the user, but the action runs only if the user is authenticated. The syntax construct starts with the term "If" followed by the description of the condition. Subsequently, the phrase must have the adverb "then", indicating the action to be performed when the condition is true.

With a *loop*, the user can describe a repetition. It starts with the word "Repeat" followed by identifiers of the actions to be repeated and the number of repetitions (e.g. "Repeat steps 5 to 10, 15 times"), or the occurrence of a stopping event.

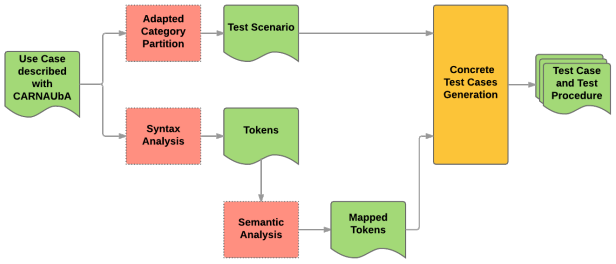

Figure 2: Overview of the test generation method.

3.2 Test Generation Method

This subsection presents the test generation method that takes as input use case description in the previouslly defined CNL. Figure 2 shows the input and output and the steps of our method: (1) Adapted Category Partition, (2) Syntax Analysis, (3) Semantic Analysis, (4) Concrete Test Case Generation; these steps are explained in the following subsections.

3.2.1 Adapted Category Partition

In order to generated the test scenarios for DSPLs, we extended PLUTO (Product Line Use Case Test Optimisation) [1], which generates test scenarios from SPL use cases. PLUTO is a black-box testing approach that adapted the Category Partition [15] method to SPLs by defining their use cases as functional units, the use case scenarios as categories and variability tags as indications for choices in a product.

Our extension of PLUTO can be summarized as follows: (i) the functional units are defined as the DSPL use cases, which should have beyond variability information the context that triggers the DSPL adaptations; (ii) besides using the use cases scenarios as categories, we also defined the variation points as categories; and (iii) in spite of using product constraints to the choices, we relate the "Necessary Context" to the choices allowing to identify the context necessary to be true to the choice be valid. Based on these adaptations, the DSPL test scenarios are then generated by combining all the possible choices from the categories identified.

3.2.2 Syntax Analysis

In the Syntax Analysis, it is performed a parsing of the use case following the restrictions in the syntax presented at the CNL Section. The result of this parsing is the tokens, which are instances of the elements typed by our language syntax; it is an identification step that marks these elements and allows for verifying whether a phrase construction is valid or not.

3.2.3 Semantic Analysis

It is at this moment that the method does the mapping of the tokens to values or commands used in the test. The verbs, for example, map into test procedures according to Table 2, which contains part of the verbs used in our approach.

Table 2: Verbs in the CNL and their correspondent commands.

Verb	Meaning	Commands	
		To the Actor	To the System
Chooses	Indicates that an option is chosen	Solo.clickOnText	–
Shows	Indicates that the system informs something to the actor	–	Solo.searchText
Types	Indicates that the actor inserts some text in the system	Solo.enterText	
Clicks	Indicates that the actor clicked in any screen point	Solo.click	–
Inserts	Indicates that the system insert something in the database	–	assertEquals
Deletes	Indicate that some information was excluded from the database	–	assertEquals
Wait	Indicates that there is a wait situation from the actor or the system response	Solo.wait	Solo.wait
Moves	Indicates that the actor have clicked and moved in the device screen	Solo.drag	–

The verbs are one of the most important elements because they indicate the actions to be performed, which will then result in the test procedure. The method associates the tokens of the verb with the tokens of the subject. With this association, the method knows what is the command to be used. For example, if the subject is an actor with the verb "chooses", it indicates that the command is "clickOnText". We use the commands following the *Robotium* [2] Android test tool (please refer to the paper's web site for the full list of commands used in this work).

The method checks if the complements of the verb are input and output. If the complement is related to an actor, the method classifies as an input (e.g., The visitor chooses the option — option is an input). Otherwise, it is classified as an output (e.g., The system shows the text — text is an output).

3.2.4 Concrete Test Case Generation

In the generation of a concrete test case to SPL, the values for the test input and their respective association with the test scenarios are set. To improve the test generation, the tester can set some values, for example, a new name for an input, or yet a restriction of the value generated as well as the maximum and minimum limits of the characters. The values used for the output are based on the rest of the use case phrase (e.g.,"selected by the visitor" in Figure 1(a)). After this step, the method generate the values. Furthermore, the method verifies which input and output are present in the test scenarios, as they may be optional according to variability information; the value of the variation point and variant are used to allow this identification. If an input or output is present, they are associated with the commands of the verbs identified in the semantic analysis. In the test cases, the input and output, with their respective values are filled. The testers can improve the tests according each application from SPL. The tests generated are executable by the FitNesse tool [3].

4. EVALUATION

[2]https://github.com/RobotiumTech/robotium
[3]www.fitnesse.org/

We evaluate our method in a pilot experiment with the goal to investigate its use during DSPL testing. The main objective is to check if the automated generation actually helps in the testing process, knowing if the automation is on the right direction or if it only creates useless artifacts on top of all the others of a DSPL. We do not evaluate the usability of CARNAUbA, we rather provide Use Cases already described with it to the Testers; this makes sense as the testers are not responsible for the definition of the Use Cases, but the requirements engineers.

Volunteers. In order to carry out the experiment, we select nine volunteers among undergraduate, masters, Ph.D. students and practitioners of four different Universities. One volunteer was an experienced software developer, three were trainees, two were master students with professional experience, two were Ph.D. students with developer experience, and one was an undergraduate student.

We divided the volunteers into two groups randomly so that each group has four or five participants. One group (Group A) is formed by one trainee, two master students, and one Ph.D. student. The other volunteers are grouped (Group B) with two trainees, one professional, one Ph.D. student and one undergraduate student.

Design. The experiment is designed to be performed as one factor (the method) and two treatments (using the method or not using the method), with Group A acting as a control group. The volunteers of Group A developed manual tests, without the support of any tool. Group B developed tests with a tool that implements our method. Both groups used the same use cases as input for test generation. The use cases are extracted from a mobile and context-aware product line, specifically for Tour Guides mobile applications, and can be found at the paper's web page.

Hypothesis. One question guided the definition of hypotheses and variables.

Q1) What is the time needed to create the tests with our method?

Answering this question will allow us to know both if the approach is actually applicable and if it brings benefits over the manual test generation process. We can now define the dependent and independent variables. The independent variable, which summarizes the study goal, is the very application of the method. There are two treatments to create the test: i) manual testing, without using the method; and ii) automatic testing, with the method implemented.

The total time spent is the dependent variable and is associated with each way to generate the tests. First, we have t_m, which is the time associated with the manual generation. The second variable is t_t that represents the time spent by the participants to create tests with our method. Therefore, the difference between these times, $\mu\_0 = t\_m - t\_t$, gives us the enhancement given by our method.

The null hypothesis (h_0): $h\_0 = \mu\_0 = 0$ of the experiment indicates that there is no significant difference between the results using the method and not using the method. The alternative hypothesis indicates that there is a significant

difference between the time of creation with the method and the time of manual creation: $h_1 = \mu_0 \neq 0$.

Operation. The volunteers create the tests for the mobile guide tour DSPL. In summary, the applications generated by this DSPL allows the users to: see information about a room of a building, browsing texts, images, videos and the list of persons who work in that room, etc. This DSPL has 23 use cases in total. We describe them by means of our CNL and make them available at the paper's web page.

This operation of the evaluation is divided into four steps: i) survey about skills and previous knowledge of the participants; ii) training; iii) execution of the experiment; and iv) survey about experiment activities.

The first activity performed by the volunteers was filling out a survey about their skills and knowledge of SPL or DSPL and about how to create a test from use case descriptions. Another information addressed by this survey was the knowledge related to technologies used in the application under test, like Android (a deep knowledge in Android was not necessary; a short training was enough).

After this, in the second activity, all volunteers participated in a training session about the creation of tests. The use cases are "Authentication" and "Show Map Caption", the later describes the instructions to show the text as the maps' caption (the map of the building which is the subject of the visit).

In sequence, the experiment was performed with three use cases. They are: "Show Text" that indicates the steps for a user to see the text associated with a room; "Change of context settings", which shows the interaction between user and system to change the settings or not according to a given context (e.g., the user chooses whether the battery power will influence or not the applications' behaviour); and the last use case is "Show the persons' data", which allows the user to see information associated with persons of an environment (e.g., profile of researchers in a laboratory). These use cases contain contextual information that is used dynamically during runtime.

The volunteers receive the use cases and we start measuring the time. They try to generate a test that can be run in the actual application and that tests the exact behaviour of the functionality and its expected output for the given input. At this point, we let them producing the test and the stop criterion for the time elapsed is if they succeeded to create a test that is in accordance to a gold standard test previously defined for the given use case. The group using our approach will need to perform adjusts to the generated tests for domain-specific parts, but we expect that those are not too daunting and this will be reflected in the results.

The last experiment step is to collect answers from the volunteers about their impressions and an evaluation of the software tests and their reusability, which we have done with manual inspections with 3 specialists (which were also present to evaluate if the volunteer had finished or not to create a valid test according to the gold standard during the experiment).

Results. Despite of the experiment being a pilot, we can have an insight about the applicability and effectiveness of our method. The results are composed by times collected and the feedback of volunteers in a second survey. The total time of each volunteer is presented in Table 3.

Table 3: Total Time for each volunteer.

	Volunteer	Total Time (min)		Volunteer	Total Time (min)
GROUP A	1	70	GROUP B	1	22
	2	53		2	19
	3	35		3	35
	4	36		4	44
	-	-		5	29
	Average	49		Average	30
	Standard Deviation	17		Standard Deviation	10

All times spent by volunteers of the group who used our method (Group B) were less, or in one case equal, to the the time spent by participants who belonged to the other group. The average time of volunteers in Group B was 30 minutes, and for the Group A, the average was 49 minutes. Naturally, carrying out the "t-test" to evaluate the results statistically with 95% of reliability, we verify that the null hypothesis is rejected. Then, the alternative hypothesis is accepted and thus, we can say that the method proposed in this work reduces the time needed to create the tests. We believe that in an experiment with more participants the results would be more expressive.

Qualitatively, we could perceive that the majority of those who participated in the manual group said that the major difficulty was the technology; the other group, said that the automated creation of tests is interesting. They have also mentioned that the method can reduce the time and effort to generate test cases, especially when there are many tests, and can help people with little or no knowledge of software testing and test scripts.

The volunteers who belonged to the Group B said that the method facilitates the creation of test scripts. We also perform an analysis of the tool that implements the method. Four participants from the Group B would use the tool and three said that the tool learning curve is low while the two others said that it is medium in a scale that ranges from very high, high, medium, low to very low. All volunteers answered that the tests created by the tool are easy to reuse.

Threats to Validity. The internal validity deals with the causal relationship between the treatment and the dependent variable.In our case, we have the fact that we have prepared ourselves the use cases with CARNAUbA. However we believe this does not incur in any biases, as the use cases are the same for both groups.The second threat is about learning capabilities of the volunteers groups, one group can learn faster than another, but the groups are distributed randomly and the assets used are the same, so we believe this effect is rather mitigated.

Threats to the external validity indicate the limits of the generalization of results. The first threat is the number of volunteers that could be greater, however the 40% enhancement let us believe that if we added more participants we would not have such an expressive decrease that it could

nullify the experiment. The second threat is the history and the context of the volunteer during the week of the experiment, but they did the experiment when they were feeling able.

5. RELATED WORK

There are several studies for automatic test case generation for traditional applications and SPL or embedded systems. However, they either do not generate test procedures or do not consider the dynamic or context-aware aspect.

In [1, 2] they generate test scenarios for traditional SPL from use cases. The difference is that, in [1], they generate only test scenarios, lacking the test procedures; while in [2] they define a method to generate test cases based on use cases with automata, but do not consider neither variability nor dynamicity.

In [14], a tool to generate and manage test scenarios for SPL was proposed. To generate tests, the authors defined a test model that generates test artifacts and its dependencies. The work of Nebut et al. [13] is presented as a method to generate test scenarios based on use cases for object-oriented embedded software. The authors define a use case simulator to detect the correctness and consistency of the use case. The work of Gois [8] defines a diagram to generate test scripts. This diagram has a graphical representation of the use case flows and allows them to associate test data with their steps as they are used. In [19], the authors define a method to generate test from use case written with a CNL, but it is for single embedded systems. We believe our work brings novelty specially by considering context information and dynamicity as part of the test generation.

6. CONCLUSIONS AND FUTURE WORK

We presented a method that generates test procedures and their input and output, taking as input use cases of Dynamic Software Product Lines described in a Controlled Natural Language, CARNAUbA, that is also defined in this paper. We run a pilot experiment in the domain of mobile and context-aware applications, verifying the applicability of the approach and then its efficiency with respect to the time spent by the user to generate a test procedure. As future work we envision the method extension to contemplate different domains of applications other than the mobile one and the execution of other statistical method to validate the results and perform the experiment with more persons. As we generate tests for every possible scenario of the DSPL, we want to consider test prioritization and the test dynamic reconfiguration as a continuation of our work.

7. REFERENCES

[1] A. Bertolino and S. Gnesi. Use case-based testing of product lines.

[2] L. Chen and Q. Li. Automated test case generation from use case: A model based approach. In *3rd IEEE International Conference on Computer Science and Information Technology (ICCSIT), 2010*, july 2010.

[3] K. Czarnecki and S. Helsen. Classification of model transformation approaches. In *Proceedings of the 2nd OOPSLA Workshop on Generative Techniques in the Context of the Model Driven Architecture*.

[4] A. K. Dey. Understanding and using context. *Personal and ubiquitous computing*, 5(1):4–7, 2001.

[5] W. Du and L. Wang. Context-aware application programming for mobile devices. In *Proceedings of the 2008 C3S2E conference*.

[6] F. Ensan, E. Bagheri, and D. Gašević. Evolutionary search-based test generation for software product line feature models. In *International Conference on Advanced Information Systems Engineering*.

[7] P. Fernandes, C. Werner, and E. Teixeira. An Approach for Feature Modeling of Context-Aware Software Product Line. *Journal of Universal Computer Science*, 17(5):807–829, 2011.

[8] F. N. B. Gois, P. Farias, and R. Oliveira. Test script diagram–um modelo para geração de scripts de testes. *Anais do IX Simpósio Brasileiro de Qualidade de Software (SBQS'10)*, pages 73–87, 2010.

[9] R. Grishman. Natural language processing. *Journal of the American Society for Information Science*.

[10] S. Hallsteinsen, M. Hinchey, S. Park, and K. Schmid. *Computer*.

[11] M. F. Johansen, Ø. Haugen, and F. Fleurey. An algorithm for generating t-wise covering arrays from large feature models. In *Proceedings of the 16th International Software Product Line Conference-Volume 1*.

[12] F. G. Marinho, F. Lima, J. a. B. F. Filho, L. Rocha, M. E. F. Maia, S. B. de Aguiar, V. L. L. Dantas, W. Viana, R. M. C. Andrade, E. Teixeira, and C. Werner. A software product line for the mobile and context-aware applications domain. In *Proceedings of the 14th International Conference on Software Product Lines: Going Beyond*.

[13] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jezequel. Automatic test generation: a use case driven approach. *Software Engineering, IEEE Transactions on*, march 2006.

[14] C. R. L. Neto. Splmt-te: A software product lines system test case tool. Master's thesis, Universidade Federal de Pernambuco, 2011. pp. 1–135.

[15] T. J. Ostrand and M. J. Balcer. The category-partition method for specifying and generating fuctional tests. *Commun. ACM*, 31(6):676–686, June 1988.

[16] G. Perrouin, S. Sen, J. Klein, B. Baudry, and Y. Le Traon. Automated and scalable t-wise test case generation strategies for software product lines.

[17] I. S. Santos, R. M. de Castro Andrade, and P. de Alcântara dos Santos Neto. A use case textual description for context aware SPL based on a controlled experiment. In *CAiSE Forum*, CEUR Workshop Proceedings.

[18] E. Uzuncaova, S. Khurshid, and D. Batory. Incremental test generation for software product lines. *IEEE Transactions on Software Engineering*.

[19] C. Wang, F. Pastore, A. Goknil, L. Briand, and Z. Iqbal. Automatic generation of system test cases from use case specifications. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ISSTA 2015, 2015.

[20] A. Wyner, K. Angelov, G. Barzdins, D. Damljanovic, B. Davis, N. Fuchs, S. Hoefler, K. Jones, K. Kaljurand, T. Kuhn, et al. On controlled natural languages: Properties and prospects. In *International Workshop on Controlled Natural Language*.

[21] T. Yue, L. C. Briand, and Y. Labiche. An automated approach to transform use cases into activity diagrams. In *European Conference on Modelling Foundations and Applications*.